Project Brief: Autoscaling LLM — เปรียบเทียบ QPS และ Queue ภายใต้ workload ต่างกัน

Oct 29, 2026 · Student brief for CENTRA 10

ภาพรวมโจทย์

โจทย์หลัก: การสเกล LLM serving ตามอัตรา request (QPS) และตามจำนวน request ที่รอคิว ให้ผลด้านคุณภาพบริการ (SLO), ความเร็วในการตอบสนองต่อโหลด และการใช้ GPU ต่างกันอย่างไรเมื่อ workload มีลักษณะแตกต่างกัน? ใช้เวลา 2 สัปดาห์ ทีม 2 คน และจัดทำโปสเตอร์สำหรับ CENTRA 10

เหตุใดจึงควรเปรียบเทียบ: สำหรับ LLM ภาระของแต่ละ request ไม่เท่ากัน โดยขึ้นกับความยาว prompt (prefill), ความยาวคำตอบ (decode) และการทำงานพร้อมกันหลาย request ของ vLLM ดังนั้น QPS หรือความยาวคิวเพียงตัวเดียวอาจไม่สะท้อนภาระได้ทุกสถานการณ์

ตัวอย่าง: ตั้งให้สเกลเมื่อเกิน 2 request ต่อวินาทีต่อเครื่อง

- คำถามสั้น 100 token ที่ 2 req/s อาจมีภาระต่ำ จึงอาจไม่จำเป็นต้องเพิ่ม replica
- เอกสารยาวหลายพัน token ที่ 2 req/s อาจทำให้ TTFT สูง แม้จำนวน request ต่อวินาทีไม่เพิ่ม

Metric vllm:num_requests_waiting ช่วยบอกจำนวน request ที่กำลังรอ แต่คิวอาจสั้นหรือเป็นศูนย์ได้แม้ GPU มีภาระสูง เพราะ vLLM กำลังประมวลผล request หลายรายการพร้อมกัน จึงต้องดู metric อื่นประกอบ

สมมติฐานเพื่อทดสอบ: queue-based scaling อาจรักษา SLO ได้ดีกว่า QPS-based scaling ใน workload แบบผสม แต่ผลขึ้นอยู่กับสภาวะของ vLLM และระยะเวลาเตรียม replica ใหม่ ทั้งสองแนวทางอาจมีข้อจำกัด จึงต้องวิเคราะห์ผลจริง ไม่ตั้งธงว่าตัวใดชนะ (SLO ในงานนี้: สัดส่วน request ที่ TTFT ≤ 2 วินาที)

SkyPilot/SkyServe เป็นระบบอ้างอิงในเครื่องมือจริงสำหรับการสเกลตาม QPS ส่วนการเปรียบเทียบหลักที่ควบคุมตัวแปรได้ดีกว่าคือ QPS-KEDA เทียบกับ Queue-KEDA บน Kubernetes เดียวกันเอกสาร SkyServe Autoscaling

สิ่งที่ต้องมีก่อนเริ่ม

พื้นฐานของทีม

- เคย deploy งานบน Kubernetes (Deployment, Service, ดู log ด้วย `kubectl`)
- เคยรัน vLLM และยิง request ผ่าน OpenAI-compatible API
- ไม่ต้องเคยใช้ SkyPilot มาก่อน ส่วนที่ 1 จะพาทำ

ทรัพยากร

รายการ	สเปกขั้นต่ำ	หมายเหตุ
K8s cluster	มี GPU node อย่างน้อย 3 ตัว	ใช้ของคณะ ไม่ต้องเปิดบัญชีคลาวด์
GPU ต่อ node	1 ตัว ขนาด 24 GB ขึ้นไป เช่น L4	จำนวน node = จำนวน replica สูงสุด

รายการ	สเปกขั้นต่ำ	หมายเหตุ
Prometheus	ติดตั้งใน cluster	เก็บ metric จาก vLLM
KEDA	ติดตั้งใน cluster	autoscaler ที่อ่าน metric จาก Prometheus ได้
Storage	PersistentVolume ประมาณ 50 GB	เก็บไฟล์โมเดลไว้ล่วงหน้า

โมเดล: ใช้โมเดลเปิดขนาดประมาณ 7B เช่น `Qwen/Qwen2.5-7B-Instruct` ไม่ต้องขอสิทธิ์บน Hugging Face และใหญ่พอให้ GPU 1 ตัวถึงจุดอิ่มตัว ถ้าใช้โมเดลเล็กเกินไป GPU จะไม่เคยเต็ม คิวไม่เกิด และการทดลองจะไม่เห็นความต่าง

ส่วนที่ 1: ทำความรู้จัก SkyPilot (วันที่ 1–3)

เป้าหมายของส่วนนี้คือ deploy vLLM ผ่าน SkyPilot ได้ และเข้าใจว่าแต่ละคำสั่งทำอะไร ทุกขั้นแก้จาก YAML ขั้นก่อนหน้าเพียงเล็กน้อย ถ้าขั้นไหนพัง ปัญหาจะอยู่ที่ส่วนที่เพิ่งเพิ่มเข้าไป

แผนที่คำสั่ง (เทียบกับ K8s ที่รู้จักแล้ว)

คำสั่ง SkyPilot	ทำอะไร	เทียบกับ K8s คร่าว ๆ
`sky launch`	เปิดเครื่องหนึ่งชุดแล้วรันงาน	สร้าง Pod ตัวเดียว
`sky serve up`	เปิดบริการหลาย replica พร้อม load balancer	Deployment + Service + HPA
`sky status` / `sky logs`	ดูสถานะ / ดู log	`kubectl get` / `kubectl logs`
`sky down` / `sky serve down`	ปิดและคืนทรัพยากร	`kubectl delete`

ระวังคำว่า "cluster": สำหรับ SkyPilot คำนี้หมายถึงเครื่องหนึ่งชุดที่มันเปิดให้ ซึ่งเมื่อใช้ K8s เป็น backend ก็คือ pod ใน K8s cluster ของคณะ ไม่ใช่ K8s cluster ใหม่

YAML ของ SkyPilot มี 4 ส่วน: `resources` (ต้องการเครื่องแบบไหน), `setup` (ติดตั้ง รันครั้งเดียว), `run` (รันงานจริง) และ `service` (ใส่เฉพาะตอนใช้ `sky serve`)

ขั้นที่ 1: ตรวจการเชื่อมต่อ

```
pip install skypilot-nightly[kubernetes]
sky check
```

ต้องเห็น Kubernetes เป็น enabled ถ้าไม่ผ่าน มักเป็นปัญหา kubeconfig (ไฟล์ตั้งค่าการเชื่อมต่อ cluster)

ขั้นที่ 2: Hello GPU

```
# hello.yaml
resources:
  infra: k8s        # เวอร์ชันเก่าใช้ cloud: kubernetes
  accelerators: L4:1
run: |
  nvidia-smi
```

```
sky launch -c hello hello.yaml
sky status
sky logs hello
sky down hello
```

เป้าหมายคือเรียนรู้วงจร launch → status → logs → down โดยยังไม่ต้องยุ่งกับ vLLM

ขั้นที่ 3: vLLM เครื่องเดียว

```
# single.yaml
resources:
  infra: k8s
  accelerators: L4:1
  ports: 8000
envs:
  MODEL_NAME: Qwen/Qwen2.5-7B-Instruct
setup: |
  pip install vllm      # ใช้เวอร์ชันปัจจุบัน  อย่าล็อกเวอร์ชันเก่าตามตัวอย่างในเว็บ
run: |
  vllm serve $MODEL_NAME --port 8000 --max-model-len 8192
```

```
sky launch -c single single.yaml
sky status --endpoint 8000 single
curl http://<endpoint>/v1/models
```

เช็กว่าได้ชื่อโมเดลกลับมา แล้วลองยิง `/v1/chat/completions` ด้วยคำถามสั้น ๆ

ขั้นที่ 4: `sky serve` แบบ 2 replica คงที่

เพิ่มส่วน `service` ต่อท้าย `single.yaml` แล้วบันทึกเป็น `serve.yaml`

```
service:
  readiness_probe:
    path: /v1/models
    initial_delay_seconds: 600
  replicas: 2
```

```
sky serve up -n llm serve.yaml
watch -n10 sky serve status llm
```

สถานะจะยังไม่ READY ระหว่างโหลดโมเดล ซึ่งอาจใช้เวลาหลายนาที เป็นเรื่องปกติ ดูความคืบหน้าได้ด้วย `sky serve logs llm 1` เมื่อ READY แล้วจะได้ endpoint เดียวที่กระจายโหลดให้ทั้ง 2 replica

ขั้นที่ 5: เปิด autoscaling

เปลี่ยน `replicas: 2` เป็น

```
  replica_policy:
    min_replicas: 1
    max_replicas: 3
    target_qps_per_replica: 2
```

```
sky serve update llm serve.yaml
```

ข้อควรระวัง: ตรวจการตั้งค่า autoscaling delay ของ SkyPilot เวอร์ชันที่ใช้จริง และบันทึกค่าไว้ เพราะอาจทำให้ไม่ทัน workload แบบ burst ที่มีช่วงเวลาสั้น

ลองยิงโหลดเบา ๆ แล้วดูว่าจำนวน replica เพิ่มขึ้นหรือไม่ การตั้งค่านี้คือ **แบบ D** ในการทดลอง

Checkpoint 1: แคปหน้าจอ `sky serve status` ตอนที่ replica เพิ่มจาก 1 เป็น 2 ขึ้นไป

ส่วนที่ 2: ตั้งระบบบน K8s (วันที่ 4–6)

ส่วนนี้สร้างแบบ B (สเกลตามอัตรา request) และแบบ C (สเกลตามคิว) บน cluster เดียวกัน และตั้งค่าระบบอื่นให้ใกล้เคียงกันมากที่สุด เนื่องจาก metric และกลไกการสเกลอาจต่างกัน ให้บันทึกการตั้งค่าที่ใช้จริง เหตุผลที่ไม่ใช้ SkyPilot เป็นคู่เทียบหลักคือ load balancer, metric และช่วงเวลาหน่วงการสเกลอาจต่างจาก K8s/KEDA

ขั้นที่ 1: เก็บโมเดลไว้ล่วงหน้า

ดาวน์โหลดโมเดลลง PersistentVolume ครั้งเดียว แล้วให้ทุก pod mount volume เดียวกัน ถ้าไม่ทำ ทุก replica ใหม่ต้องดาวน์โหลดเองหลายนาที และ cold start (เวลาตั้งแต่สั่งเพิ่มเครื่องจนรับ request ได้) จะบังผลการทดลองทั้งหมด

ขั้นที่ 2: Deployment ของ vLLM

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vllm
spec:
  replicas: 1
  selector:
    matchLabels: {app: vllm}
  template:
    metadata:
      labels: {app: vllm}
    spec:
      containers:
      - name: vllm
        image: vllm/vllm-openai:latest
        args: ["--model", "Qwen/Qwen2.5-7B-Instruct", "--max-model-len", "8192"]
        env:
        - {name: HF_HOME, value: /models}
        ports:
        - containerPort: 8000
        resources:
          limits: {nvidia.com/gpu: 1}
        readinessProbe:
          httpGet: {path: /health, port: 8000}
          periodSeconds: 5
        volumeMounts:
        - {name: models, mountPath: /models}
      volumes:
      - name: models
        persistentVolumeClaim: {claimName: model-cache}
```

สร้าง Service ชี้ไปที่ port 8000 แล้วให้ Prometheus เก็บ metric จาก `/metrics` ของทุก pod (ใช้ ServiceMonitor หรือ PodMonitor ถ้าติดตั้ง Prometheus Operator)

ขั้นที่ 3: เช็กชื่อ metric จริง

```
kubectl port-forward deploy/vllm 8000:8000
curl -s localhost:8000/metrics | grep -E "num_requests|request_success|kv_cache"
```

เช็กชื่อ metric จริงจาก /metrics ของ vLLM เพราะอาจต่างกันตามเวอร์ชัน จดเวอร์ชันและชื่อที่พบ โดยเก็บ waiting requests, running requests และ KV-cache usage หากมี (เช่น vllm:num_requests_waiting, vllm:num_requests_running, vllm:kv_cache_usage_perc)

ขั้นที่ 4: แบบ B สเกลตาม QPS

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: vllm-qps
spec:
  scaleTargetRef: {name: vllm}
  minReplicaCount: 1
  maxReplicaCount: 3
  pollingInterval: 15
  advanced:
    horizontalPodAutoscalerConfig:
      behavior:
        scaleDown:
          stabilizationWindowSeconds: 120
  triggers:
  - type: prometheus
    metadata:
      serverAddress: http://prometheus.monitoring:9090
      query: sum(rate(vllm:request_success_total[1m]))
      threshold: "2"
```

คำสั่งนี้ใช้ vllm:request_success_total ซึ่งนับ request ที่ประมวลผลสำเร็จแล้ว (completed QPS) ไม่ใช่ incoming QPS โดยตรง จึงให้เก็บอัตรา request ที่ส่งเข้าระบบจาก load generator เพิ่มด้วยเพื่อใช้ประกอบการวิเคราะห์ ห้ามสรุปว่าตัววัดสองชนิดเหมือนกัน และให้จดการตั้งค่า KEDA/HPA จริง

`stabilizationWindowSeconds` คือเวลาที่รอก่อนลดเครื่อง กันไม่ให้ระบบเพิ่มลดเครื่องสลับไปมา

ขั้นที่ 5: แบบ C สเกลตามคิว

คัดลอก ScaledObject ของแบบ B แล้วเปลี่ยนแค่ชื่อ query และ threshold

```
      query: sum(vllm:num_requests_waiting)
      threshold: "2"
```

เท่ากับตั้งเป้าว่าเฉลี่ยแล้วแต่ละเครื่องมี request รอคิวไม่เกิน 2 ตัว **ใช้ทีละแบบเท่านั้น** ห้ามมี ScaledObject สองตัวชี้ Deployment เดียวกันพร้อมกัน ลบตัวเก่าก่อนสร้างตัวใหม่ทุกครั้ง

ขั้นที่ 6: จูน threshold

ก่อนวัดจริง ให้รันแบบ A (replica คงที่ 1 ตัว) เพิ่มโหลดทีละขั้นจน TTFT เริ่มเกิน 2 วินาที จด completed QPS, incoming QPS และความยาวคิว เพื่อใช้เป็นจุดตั้งต้นในการจูน threshold ทั้ง B และ C ด้วยหลักเกณฑ์เดียวกัน โดยบันทึก threshold และเหตุผลที่เลือก

Checkpoint 2: Grafana dashboard แสดง replica, waiting/running requests, อัตรา request และ KV-cache usage (หากมี) ตามเวลา พร้อมจดชื่อ metric ที่ใช้จริง

ส่วนที่ 3: การทดลอง (วันที่ 7–10)

การทดลองมี 4 การตั้งค่า × 4 workload × 3 รอบ รวม 48 รอบ รอบละประมาณ 10–12 นาที ใช้เวลารันรวมราว 10 ชั่วโมง เขียน script ให้รันต่อกันอัตโนมัติได้

การตั้งค่าที่เทียบ

แบบ	สเกลตาม	หน้าที่
A. Fixed	ไม่สเกล replica คงที่ 1 ตัว	baseline
B. QPS	Completed QPS จาก vLLM (KEDA); เก็บ incoming QPS ประกอบ	ตัวแทนแนวคิดของ SkyPilot
C. Queue	`vllm:num_requests_waiting` ต่อ replica (KEDA)	แบบที่เสนอ
D. SkyPilot	`sky serve` ตามส่วนที่ 1 ขั้นที่ 5	ตัวอ้างอิงจากเครื่องมือจริง (ไม่ใช่คู่เทียบควบคุมสมบูรณ์)

ตัวแปรที่ต้องควบคุม: โมเดล/เวอร์ชัน vLLM, GPU, max-model-len, replica ต่ำสุด 1 สูงสุด 3, workload ที่ส่งเข้า และสถานะเริ่มต้น 1 replica ที่พร้อมใช้งาน บันทึกเวลารอการสเกล/ลด replica และเวลาที่ replica ใหม่พร้อมรับ request ของแต่ละระบบ โดยเฉพาะ SkyPilot ซึ่งอาจใช้ delay ไม่เท่ากับ KEDA

Workload 4 แบบ

Workload	ลักษณะ	สิ่งที่ทดสอบ
1. Short	input ~100 token, output ~100 token	QPS สะท้อนภาระได้ดีเมื่อทุก request เบาเท่ากัน
2. Long	input ~6,000 token, output ~200 token	QPS สะท้อนภาระได้ดีเมื่อทุก request หนักเท่ากัน
3. Mixed	short 80% ปนกับ long 20%	ผลของ request ยาว–สั้นปะปนต่อสัญญาณสเกล
4. Bursty	โหลดต่ำ 3 นาที → สูง 4 เท่า 3 นาที → ต่ำ 3 นาที	ความไวในการตอบสนองและการลดเครื่อง

สร้างโหลดด้วย `vllm bench serve` แบบ random dataset ตัวอย่างสำหรับ workload 1

```
vllm bench serve \
  --backend openai-chat --endpoint /v1/chat/completions \
  --base-url http://<service> --model Qwen/Qwen2.5-7B-Instruct \
  --dataset-name random --random-input-len 100 --random-output-len 100 \
  --request-rate 4 --num-prompts 2400 \
  --save-result --result-filename short_B_run1.json
```

สำหรับ workload 3 ให้รัน 2 คำสั่งพร้อมกัน คำสั่งหนึ่งเป็น short อีกคำสั่งเป็น long โดยตั้ง `--request-rate` ให้ได้สัดส่วน 80:20 สำหรับ workload 4 ให้รันสามช่วงต่อกันด้วย request rate ต่างกัน ชื่อ flag อาจต่างไปตามเวอร์ชัน ให้เช็กด้วย `vllm bench serve --help`

ข้อสำคัญ: ใช้ workload และ offered request rate เดียวกันสำหรับทุกแบบในแต่ละกรณี เลือกโหลดที่ทำให้แบบ A เริ่มไม่ผ่าน SLO เพื่อเห็นความแตกต่าง แต่ตรวจให้แน่ใจว่าไม่มีความผิดพลาดจากการสร้างโหลดหรือทรัพยากรอื่นคอขวด

สิ่งที่วัด

ตัวชี้วัด	ได้จากไหน	ความหมาย
TTFT p50 / p99	ไฟล์ผลของ `vllm bench serve`	ค่ากลาง และค่าของ request ที่ช้าที่สุด 1%
SLO attainment	คำนวณจาก TTFT ราย request	สัดส่วน request ที่ TTFT ≤ 2 วินาที
GPU-hours	จำนวน replica × เวลา จาก Prometheus	ตัวแทนต้นทุน
เวลาตอบสนอง	กราฟจำนวน replica ตามเวลา	จากโหลดพุ่งจนเครื่องใหม่รับ request ได้
Cold start	log ของ pod	เวลาตั้งแต่ pod เริ่มจน readiness ผ่าน รายงานแยก
Runtime metrics (เสริม)	Prometheus / vLLM / load generator	incoming QPS, running requests และ KV-cache usage (ถ้ามี) เพื่ออธิบายเหตุการณ์
Replica ready / scaling delay	Kubernetes / SkyPilot events และ logs	ช่วงตั้งแต่ตรวจพบโหลดจน replica ใหม่พร้อมใช้งาน

สิ่งที่ต้องตอบจากผลจริง: แบบใดรักษา TTFT SLO ได้ดีกว่า ใช้ GPU-hours ต่างกันเท่าใด และเหตุใด? วิเคราะห์กรณี queue สั้นแต่ TTFT สูง, ความต่างระหว่าง incoming/completed QPS และผลของเวลาที่ replica ใหม่พร้อมใช้งาน หาก B และ C ให้ผลใกล้กัน ก็เป็นผลที่ยอมรับได้เมื่อมีหลักฐานรองรับ

Checkpoint 3: ผลรอบแรกของ workload 3 ครบทั้ง 4 แบบ ส่งให้อาจารย์ดูก่อนรันที่เหลือ

ไทม์ไลน์ 2 สัปดาห์

โปรเจกต์มี 4 checkpoint ทุกตัวต้องผ่านก่อนไปขั้นถัดไป ถ้าติดเกินหนึ่งวันให้แจ้งอาจารย์ทันที ไม่ต้องรอถึงวันส่ง

1. **วันที่ 1–3 — ทำความรู้จัก SkyPilot (ส่วนที่ 1)**
   - Checkpoint 1: `sky serve` สเกลจาก 1 เป็น 2 replica ได้
2. **วันที่ 4–6 — ตั้งระบบบน K8s (ส่วนที่ 2)**
   - Checkpoint 2: dashboard แสดง replica, waiting/running requests และอัตรา request (รวม KV-cache usage หากมี)
3. **วันที่ 7–10 — รันการทดลอง (ส่วนที่ 3)**
   - Checkpoint 3: ผล workload 3 รอบแรกครบ 4 แบบ ให้อาจารย์ดูก่อนรันที่เหลือ
4. **วันที่ 11–14 — วิเคราะห์ผลและทำโปสเตอร์**
   - Checkpoint 4: ร่างโปสเตอร์ส่งให้อาจารย์ตรวจอย่างน้อย 2 วันก่อนส่งจริง

การแบ่งงานในทีม 2 คน: ทั้งสองคนทำส่วนที่ 1 ด้วยกันเพื่อให้เข้าใจภาพเดียวกัน จากนั้นแยกกันดังนี้

ช่วง	คนที่ 1: ฝั่งระบบ	คนที่ 2: ฝั่งการทดลอง
วันที่ 1–3	ส่วนที่ 1 ด้วยกัน	ส่วนที่ 1 ด้วยกัน
วันที่ 4–6	Deployment, Prometheus, KEDA แบบ B และ C	script สร้างโหลด 4 workload และ script เก็บผลจาก Prometheus
วันที่ 7–10	ดูแลระบบระหว่างรัน สลับการตั้งค่า แก้ปัญหา	รันการทดลองตามคิว ตรวจว่าผลครบทุกรอบ
วันที่ 11–14	เขียนส่วนการตั้งค่าในรายงานและ README	วิเคราะห์ผล ทำกราฟ และร่างโปสเตอร์

แบบ D (SkyPilot) ใช้การตั้งค่าจากส่วนที่ 1 ขั้นที่ 5 ได้เลย ไม่ต้องมีคนดูแลแยก การรัน 48 รอบต้องเขียน script ให้รันต่อกันอัตโนมัติ ไม่อย่างนั้นสองคนจะไม่ทันในวันที่ 7–10

งานที่ต้องส่งและเกณฑ์ประเมิน

งานที่ต้องส่ง

- ☐ GitHub repository: YAML ทุกไฟล์, script สร้างโหลด, script วิเคราะห์ผล และ README ที่บอกวิธีรันซ้ำได้
- ☐ ไฟล์ผลดิบทั้ง 48 รอบ (JSON จาก vllm bench serve, จำนวน replica และ metrics จาก Prometheus; เก็บ offered QPS, waiting/running requests, KV-cache usage ถ้ามี และเวลา replica ready)
- ☐ รายงานสั้น 4–6 หน้า: คำถาม วิธีทดลอง ผลเปรียบเทียบ และเหตุผลของผลลัพธ์ พร้อมเวอร์ชัน vLLM, SkyPilot, KEDA, threshold และ scaling delays ที่ใช้จริง
- ☐ โปสเตอร์สำหรับ CENTRA 10 (ภาษาอังกฤษ)

กราฟที่โปสเตอร์ต้องมีอย่างน้อย

1. SLO attainment เทียบกับ GPU-hours ของทุกแบบใน workload 3 เป็นกราฟที่ตอบคำถามหลักได้ในภาพเดียว
2. จำนวน replica ตามเวลา ซ้อนกับ waiting requests ของแบบ B และ C ใน workload 4 พร้อมทำเครื่องหมายช่วงเริ่ม burst และเวลาที่ replica ใหม่พร้อมใช้งาน
3. TTFT p99 ของทุกแบบในทั้ง 4 workload

เกณฑ์ประเมิน

หัวข้อ	น้ำหนัก	สิ่งที่ดู
การออกแบบการทดลอง	25%	ล็อกตัวแปรครบ เทียบอย่างยุติธรรม จูน threshold มีเหตุผล
ความถูกต้องและการทำซ้ำได้	20%	คนอื่นรันตาม README แล้วได้ผลใกล้เคียง มีผล 3 รอบต่อเงื่อนไข
การวิเคราะห์	30%	อธิบายได้ว่าทำไมผลออกมาแบบนั้น รวมถึงผลที่ไม่ตรงสมมติฐาน
การสื่อสาร	25%	โปสเตอร์ตอบคำถามหลักได้ใน 30 วินาที กราฟอ่านง่าย มีหน่วยครบ

ปัญหาที่พบบ่อยและกฎการใช้ทรัพยากร

อาการ	สาเหตุที่เป็นไปได้	วิธีแก้
`sky serve status` ไม่ขึ้น READY นานหลายนาที	กำลังโหลดโมเดล	ดู `sky serve logs` ถ้ายังโหลดอยู่ให้รอ ปกติ
pod ล้มตอนเริ่มเพราะหน่วยความจำ GPU ไม่พอ	context ยาวเกินไป	ลด `--max-model-len` หรือปรับ `--gpu-memory-utilization`
ทุกแบบผ่าน SLO หมด ไม่เห็นความต่าง	โหลดเบาเกินไป	เพิ่ม request rate จนแบบ A ไม่ผ่านชัดเจน
คิวเป็นศูนย์ตลอดแม้ TTFT สูง	vLLM รับ request เข้าประมวลผลพร้อมกันหมด	ลด `--max-num-seqs` หรือลองใช้ `vllm:num_requests_running` เป็น metric เสริม
replica เพิ่มลดสลับไปมา	เวลารอก่อนลดเครื่องสั้นเกินไป	เพิ่ม `stabilizationWindowSeconds`
KEDA ไม่สเกลเลย	query ใน Prometheus คืนค่าว่าง	ลองรัน query ใน Prometheus UI ก่อน และเช็กชื่อ metric ตามส่วนที่ 2 ขั้นที่ 3

กฎการใช้ทรัพยากร

- ทำเสร็จแล้วต้อง `sky down` หรือ `sky serve down` และลบ ScaledObject ทุกครั้ง GPU ที่ลืมคืนจะทำให้ทีมอื่นใช้ไม่ได้
- ห้ามใส่ Hugging Face token หรือ kubeconfig ลงใน repository ใช้ Secret หรือตัวแปรสภาพแวดล้อมแทน
- จองช่วงเวลารันการทดลองใหญ่ (วันที่ 7–10) กับอาจารย์ล่วงหน้า เพื่อไม่ให้ทีมอื่นใช้ GPU ชนกันจนผลเพี้ยน

แหล่งอ้างอิง

- vLLM — Deploying with SkyPilot
- SkyPilot — SkyServe Autoscaling
- SkyPilot — Service YAML spec